

CLI Agent 构造手册

拆开四个真实项目的引擎盖，看看“终端里的智能体”到底是怎么拼出来的——原理、六个必备构件、以及一份能自己动手跑起来的最小实现。

案例源码：

[claudecode](#)[openclaw-2026.3.23](#)[pi](#)[hermes-agent](#)

整理日期 2026-07-12

00 · 定义

CLI Agent 到底是什么

一句话：CLI Agent = 一个跑在终端里、能反复“思考 → 调用工具 → 观察结果”直到任务做完的程序，而不是一问一答的聊天框。区别不在“用了大模型”，而在有没有那个循环。

聊天机器人

输入 → 模型 → 输出文字

一次性、只能吐文字，看不见你的文件系统，也碰不了你的 shell。

≠

CLI AGENT

输入 → 模型 ⇄ 工具(读写文件/跑命令/搜索) ⇄ 最终答案

活在你的文件系统和 shell 里，能循环很多轮，直到模型自己觉得任务完成。

01 · 核心原理

所有 CLI Agent 的心脏：Agent Loop

不管是几万行的 `claudecode`，还是几十行的最小实现，本质都是同一个循环：

① 发消息给模型

把「系统提示词 + 历史消息 + 可用工具清单」一起发给 LLM。

② 模型做判断

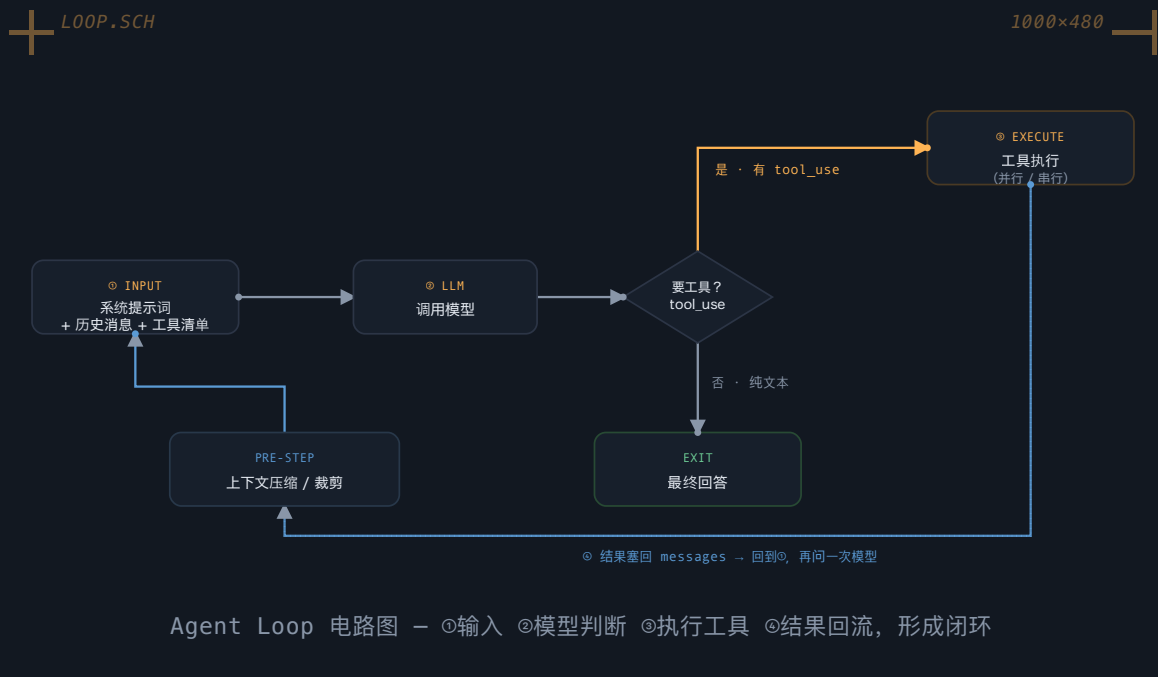
要么直接吐文本（任务结束），要么要求调用某个工具（继续）。

③ 真正执行工具

在你的机器上跑这个函数（读文件 / 跑命令 / 搜索...），拿到结果。

④ 结果塞回消息列表，回到①

模型在下一轮"看到"工具的执行结果，据此继续决策，直到不再要求工具，或触发轮次上限。



四个项目里真正对应这个循环的入口函数：

- cc** `queryLoop()` · `query.ts` —— 一个 `while(true)` 的 async generator, 每个 chunk 都 yield 出来, UI 增量渲染。
- oc** 自己不写循环, 委托给外部 SDK `pi-agent-core`, 编排入口是 `runEmbeddedPiAgent()` · `pi-embedded-runner/run.ts`。
- pi** `runLoop()` · `agent-loop.ts` —— 外层 while 处理"用户中途追加消息", 内层 while 处理"还有没有工具调用"。
- hs** `run_conversation()` · `agent/conversation_loop.py` —— `while api_call_count < max_iterations` (默认 90 轮)。

02 · 拆解

把 Agent Loop 拆成六个可以单独理解的构件

循环本身只有几行, 难的是"循环转起来不崩"所需要的周边系统。下面按你自己动手实现时会遇到的顺序, 逐个拆解——每个都回答一个问题: 这东西为什么存在?

01

循环引擎 (Loop Engine)

所有智能都发生在"调模型"这一步, 循环引擎本身很笨——它只负责"再问一次"。真正决定 agent 好不好用的, 是这个循环要不要处理流式输出、要不要并行跑多个工具、要不要在中途允许用户插话。循环笨, 周边不笨。

02

工具接口与注册表 (Tool Registry)

LLM 不会真的"调用"任何东西——它只是吐出一段结构化文本(工具名 + 参数), 是你的程序读懂这段文本、在注册表里找到对应函数、真正执行、再把结果写回对话。工具的 JSON Schema 就是喂给模型的"菜单", 菜单设计得好不好, 直接决定模型点菜点不点得准。

03

系统提示词的动态拼装 (System Prompt Assembly)

系统提示词不是一次写死、永远不变的字符串, 它更像每轮现拼的三明治——层次越拆得清楚, 越容易做提示词前缀缓存 (prefix cache) 省钱:

L1 产品身份 / 工具使用规范（几乎不变，最利于缓存命中）

L2 运行环境信息（cwd、git 状态、操作系统、当前时间）

L3 项目 / 用户专属规则文件（`CLAUDE.md` / `AGENTS.md` / `SOUL.md` ...）

L4 记忆快照 / 本轮追加指令（最易变，放最后）

04

上下文压缩（Context Compaction）

对话开得越久，历史消息越多，而模型的上下文窗口是有限的。四个项目不约而同用了同一套分级策略，越往后成本越高、触发越少：

STAGE 1

截断/落盘超大工具输出
比如读了几万行的文件

→

STAGE 2

轻量合并 / 剪枝
去掉重复、过期的历史片段

→

STAGE 3

触发阈值 → 模型自我总结
用摘要替换原始历史

05

权限与审批（Permission Gate）

凡是会"动手"的工具（写文件、跑命令、发消息）都要过一道闸门：按规则直接判定，判不出来就弹出来问用户一句。这道闸门放在哪、松紧到什么程度，决定了 agent 是"放心用"还是"提心吊胆用"。典型判定顺序：



命中黑名单规则

直接拒绝，不问模型也不问用户



命中"需询问"规则 / 工具自检未通过

弹窗给用户，等待 允许一次 / 本次会话 / 始终允许 / 拒绝



命中白名单 / 只读安全操作

直接放行，无需打断用户

06

进阶能力（生产级的加分项）

循环 + 工具 + 提示词 + 压缩 + 权限，五件套凑齐就是一个能用的 agent 了。再往上，是让它"敢在生产里跑"的加分项——是否流式、能不能并行、能不能拆子任务、能不能沙箱化、接不接得了多个模型和多个消息渠道。见下方矩阵。

一张表看四个项目怎么落地这六个构件（点位为函数名 + 文件，可去仓库里对照原文）：

构件	CLAUDECODE	OPENCLAW	PI	HERMES-AGENT
01 循环引擎	<code>queryLoop()</code> query.ts	委托 <code>pi-agent-core</code> runEmbeddedPiAgent() attempt.ts	<code>runLoop()</code> agent-loop.ts	<code>run_conversation()</code> conversation_loop.py
02 工具接口	<code>Tool<Input,Output></code> Tool.ts + getAllBaseTools().tools.ts	<code>AnyAgentTool</code> createOpenClawCodingTools(). pi-tools.ts	<code>ToolDefinition</code> (typebox) coding-agent/ core/tools/ *.ts	<code>registry.register()</code> tools/ registry.py (自动发现)
03 系统提示词	<code>buildEffectiveSystemPrompt()</code> systemPrompt.ts + CLAUDE.md	<code>buildAgentSystemPrompt()</code> system-prompt.ts + AGENTS.md/ SOUL.md	<code>buildSystemPrompt()</code> system-prompt.ts + AGENTS.md	<code>build_system_prompt_parts()</code> stable/ context/ volatile 三层 + 前缀缓存
04 上下文压缩	budget→ <code>microcompact</code> services/ compact/*	<code>truncate</code> (分段摘要) agents/ compaction.ts	<code>compact</code> , <code>shouldCompact</code> → <code>generateSummary→compact()</code> harness/ compaction/ compaction.ts	<code>ContextCompressor</code> 五步算法 context_compressor.py
05 权限审批		ExecSecurity 三级 +	官方声明：无内置权限系统，审	

	<code>hasPermissions</code> <code>ToUseToolInner</code> <code>()</code> 五级判定 <code>permissions.ts</code>	Gateway 两阶段 RPC <code>exec-</code> <code>approvals.ts</code>	批是扩展可选钩子 README.md + <code>extensions/</code> <code>types.ts</code>	<code>check_all_comm</code> <code>and_guards()</code> + 交互式选择 <code>tools/</code> <code>approval.py</code>
--	---	--	--	--

进阶能力矩阵——谁做了、谁没做：

能力	CLAUDECODE	OPENCLAW	PI	HERMES-AGENT
流式输出	✓ AsyncGenerator 逐 chunk	✓ 继承底层 streamFn	✓ EventStream 事件流	✓ 流式采集
并行工具调用	✓ 只读并行 / 写操作串行	✓ 依赖底层 SDK	✓ <code>executeToolCallsParallel</code>	✓ 并发/串行两种 执行器
子代理 Sub-agent	✓ AgentTool 递归调用自身	✓ <code>sessions_spawn</code> (限深度)	— 未发现	— 未发现
沙箱隔离	— 依赖 hooks 自行接入	✓ Docker, 后 端可插拔	— 明确建议自 行容器化	✓ local/ docker/ssh/ daytona/modal
多模型 Provider	— 绑定 Claude API	✓ 18+ providers	✓ 30+ providers (独立 ai 包)	— 依赖网关配 置
多渠道接入 (IM 等)	—	✓ Gateway + 82 个插件	—	✓ Telegram/ Slack/微信/ QQ/ iMessage...

四个项目速览

同样是"CLI agent", 四份项目的定位其实完全不同——从"编辑器里的编码助手"到"全渠道消息平台", 跨度很大。

claudecode

Anthropic 官方 CLI 编码助手

- TypeScript 单体应用, 围绕"编码任务"深度打磨
- 工具按需加载 (ToolSearch), 压低系统提示词体积
- Stop Hooks / PreToolUse 等生命周期钩子, 用户可编程干预
- 子代理即"递归调用自己的 query()", 架构自相似

语言: TypeScript 定位: 单人编码助手

openclaw

多渠道消息驱动的 agent 网关

- 自己不写 agent loop, 委托给独立 SDK `pi-agent-core`, 专注"编排层"
- Gateway 统一网关 + 82 个扩展插件 (渠道/模型/工具)
- Docker 沙箱可插拔, `cron/` 支持定时自任务
- 危险操作走网关两阶段 RPC 审批, 多端 (macOS/iOS/Web) 可响应

语言: TypeScript monorepo 定位: 团队/个人全渠道助理

pi

分层良好的 agent 运行时 + CLI

- 五个包分工清晰: `ai` (LLM 抽象) / `agent` (运行时) / `coding-agent` (CLI) / `tui` / `orchestrator`
- 自研差分渲染 TUI, 不依赖 Ink/blessed
- 30+ 模型 provider, 事件流架构, UI 增量订阅渲染
- 明确声明不做权限系统, 把裁量权交给使用者或容器

语言: TypeScript monorepo 定位: 可复用 agent 运行时

hermes-agent

全渠道 agent 平台 + ACP 协议适配

- Python, 工具注册表自动发现 `tools/*.py`, 无需手工登记列表
- 系统提示词 stable/context/volatile 三层, SQLite 缓存, 复用 LLM 前缀缓存
- gateway/platforms 打通 Telegram / Slack / 微信 / QQ / iMessage 等
- 支持 ACP (Agent Client Protocol), 可被 Zed 等编辑器当作 agent 驱动

语言: Python 定位: 全渠道个人/团队助理

最小可行 Agent：不到 60 行

抛开工程细节，一个“能跑”的 CLI agent 用不了多少代码。对照着看，六个构件里真正躲不掉的只有①循环 ②工具表 ③LLM 调用三件——其余都是“规模大了以后不得不加”的加固。

伪代码：一个能跑的最小 CLI agent

```
TOOLS = {
    "read_file": {"schema": {...}, "run": read_file},
    "bash":      {"schema": {...}, "run": run_bash},
}

def agent_loop(user_input, system_prompt):
    messages = [{"role": "user", "content": user_input}]

    for turn in range(MAX_TURNS):
        response = llm.call(
            system=system_prompt,
            messages=messages,
            tools=[t["schema"] for t in TOOLS.values()],
        )
        messages.append(response.to_message())

        if not response.tool_calls:
            return response.text          # ① 模型直接回答 → 循环结束

        for call in response.tool_calls: # ② 一次可能要求多个工具
            tool = TOOLS[call.name]
            if is_dangerous(call) and not ask_user_permission(call):
                result = "denied by user"
            else:
                result = tool["run"](**call.arguments)
            messages.append(tool_result_message(call.id, result))
        # ③ 结果已塞回 messages，回到循环顶部，让模型“看到”结果

    return "达到最大轮次，任务未完成"
```

- ① 结束条件只有一个：模型这一轮 **没有** 要求任何工具调用。
- ② 一次响应里可能带多个工具调用（比如同时读两个文件），要循环处理，别只取第一个。
- ③ 整个 agent 没有"隐藏状态"——

`messages`

这个列表本身就是全部状态，模型的"记忆"就是把它整段重新发一遍。

把这 60 行放进生产，你需要：上下文超限时压缩（04）、危险操作弹窗确认（05）、流式展示进度、工具执行沙箱化——这正是上一节六大构件存在的原因，而不是额外的"炫技"。

05 · 清单

从 MVP 到生产级：加固清单

按"什么时候会痛"来排序，痛了再加，不必一开始就全都做。



对话开长了会报错 / 变慢 → 加上下文压缩

先截断超大工具输出，再轻量剪枝，最后才用模型自我总结（构件 04）。



敢让它碰真实文件系统 / 生产环境 → 加权限闸门

写操作、删除、执行命令都要过一道 允许/询问/拒绝（构件 05）。



用户等得着急 → 加流式输出

把 LLM 的 token 和工具执行进度实时吐给终端，而不是等全部跑完再显示。



一次任务要读/搜很多文件 → 加并行工具执行

只读、无副作用的工具可以并发跑；有副作用的（写文件）必须串行。



任务太复杂，一个循环装不下 → 加子代理

把子任务委派给一个新的、独立上下文的 agent 循环，做完汇报结果（如 Task / sessions_spawn）。



要在多用户 / CI / 不受信输入下跑 → 加沙箱

工具执行换成容器/远程执行后端，而不是直接跑在宿主机上。



想换模型，或想接入 IM → 加 provider 抽象层 + 消息网关

把“调用哪个模型”“从哪个渠道收发消息”从核心循环里剥离出去。

```
claudecode → /studio/claudecode  
openclaw → /studio/openclaw-2026.3.23  
pi → /studio/pi  
hermes-agent → /studio/hermes-agent
```

由 Claude Code 协助整理 · 2026-07-12